

SOLVED SAMPLE TEST :

Fundamentals of Quantum Computing Using Qiskit v2.X Developer

Python version

```
In [1]: from platform import python_version  
  
print(python_version())
```

3.13.5

Qiskit Version

```
In [2]: import qiskit  
  
print(f"Qiskit version: {qiskit.__version__}")
```

Qiskit version: 2.1.1

```
In [3]: from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, AncillaRegister  
  
from IPython.display import Image  
  
from qiskit.quantum_info import Statevector  
from qiskit.visualization import plot_histogram, plot_state_qsphere, plot_bloch_multivector  
  
from qiskit.quantum_info import Pauli  
from qiskit.circuit import QuantumCircuit, Parameter, ParameterExpression  
import numpy as np
```

Section 1: Perform quantum operations

1. Which one of the following code fragments will generate the given output?

```
[[ 1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]]
```

- a. `p = Pauli('IZ')`
`print(p.to_matrix())`
- b. `p = Pauli('-II')`
`print(p.to_matrix())`
- c. `p = Pauli('-ZI')`
`print(p.to_matrix())`
- d. `p = Pauli('ZZ')`
`print(p.to_matrix())`

REFERENCE: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Pauli

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

For example 'XYZ'

-- Represents $X \otimes Y \otimes Z$

--'Z' on qubit-0

--'Y' on qubit-1, and

--'X' on qubit-2.

Option a:

```
In [4]: p = Pauli('IZ')
print(p.to_matrix())
```

```
[[ 1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]]
```

Option b:

```
In [5]: p = Pauli('-II')
print(p.to_matrix())
```

```
[[ -1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j -1.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]]
```

Option c:

```
In [6]: p = Pauli('-ZI')
print(p.to_matrix())
```

```
[[-1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  1.-0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j  1.-0.j]]
```

Option d:

```
In [7]: p = Pauli('ZZ')
        print(p.to_matrix())
```

```
[ [ 1.+0.j  0.+0.j  0.+0.j  0.+0.j]
  [ 0.+0.j -1.+0.j  0.+0.j  0.+0.j]
  [ 0.+0.j  0.+0.j -1.+0.j  0.+0.j]
  [ 0.+0.j  0.+0.j  0.+0.j  1.+0.j]]
```

ANSWER: a

2. Applying the Qiskit TGate to a qubit in state $|1\rangle$ introduces which global phase?

- a. $\pi/2$ phase
- b. $-\pi/2$ phase
- c. $-\pi/4$ phase
- d. $\pi/4$ phase

```
In [8]: qc = QuantumCircuit(1)

        # By default every qubit is initialized to  $|0\rangle$  state
```

```
# Initialize Qubit to state |1> using X gate
qc.x(0)

# apply T gate
qc.t(0)

qc.draw(output="mpl")
```

Out [8]:



Matrix Representation:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.TGate>

- **T gate is a phase gate**
- Introduces a phase shift of $\pi/4$ (or 45 degrees) to the $|1\rangle$ state of a qubit
- Affects the phase of the qubit's state, leaving the $|0\rangle$ state unchanged
- This phase shift corresponds to a rotation of $\pi/4$ radians (or 45 degrees) around the Z-axis of the Bloch sphere.
- the global phase is a phase factor applied to the entire quantum state of a circuit
- doesn't affect measurement outcomes
- It can be represented as $e^{i\theta}$, where θ is the phase angle

```
In [9]: state= Statevector(qc)
state.draw(output='latex')
```

Out[9]:

$$\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}i}{2}\right)|1\rangle$$

In [10]: `from qiskit.circuit.library import TGate`

Create an instance of the TGate

`t_gate = TGate()`

Get the unitary matrix of the TGate

`unitary_matrix = t_gate.to_matrix()`

`print("Unitary Matrix of T gate:")`

`print(unitary_matrix)`

Unitary Matrix of T gate:

```
[[1.      +0.j      0.      +0.j      ]
 [0.      +0.j      0.70710678+0.70710678j]]
```

Matrix Representation:

$$Tdg = \begin{pmatrix} 1 & 0 \\ 0 & e^{-i\pi/4} \end{pmatrix}$$

REFERENCE : <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.TdgGate>

T-dagger gate reverses the phase shift introduced by the T gate.

It introduces a **phase shift of - pi/4** to the $|1\rangle$ component, leaving the $(|0\rangle)$ component unchanged.

Matrix Representation:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

REFERENCE : <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.SGate>

Another Phase gate (S gate)

- known as the $P(\pi/2)$ gate
- introduces a global phase shift of $\pi/2$
- single-qubit gate that induces a $\pi/2$ phase shift on the $|1\rangle$ state
- It is equivalent to a rotation of $\pi/2$ radians around the Z-axis of the Bloch sphere.
- S gate is equivalent to $R_z(\pi/2)$.

Matrix Representation:

$$Sdg = \begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}$$

REFERENCE : <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.SdgGate>

It induces a. $-\pi/2$ phase

ANSWER: d
phase shift of $\pi/4$ (or 45 degrees) to the $|1\rangle$ state of a qubit

3. Given the following code fragment, what is the approximate probability that a measurement would result in a bit value of 1?

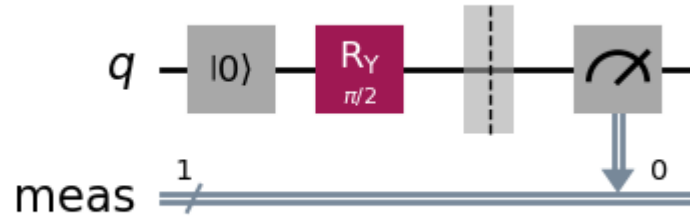
```
from qiskit import QuantumCircuit
import numpy as np
```

```
qc = QuantumCircuit(1)
qc.reset(0)
qc.ry(np.pi / 2, 0)
qc.measure_all()
```

- a. 0.8536
- b. 1.0
- c. 0.1464
- d. 0.5

```
In [11]: qc = QuantumCircuit(1)
          qc.reset(0)
          qc.ry(np.pi / 2, 0)
          qc.measure_all()
          qc.draw(output="mpl")
```


Out[11]:



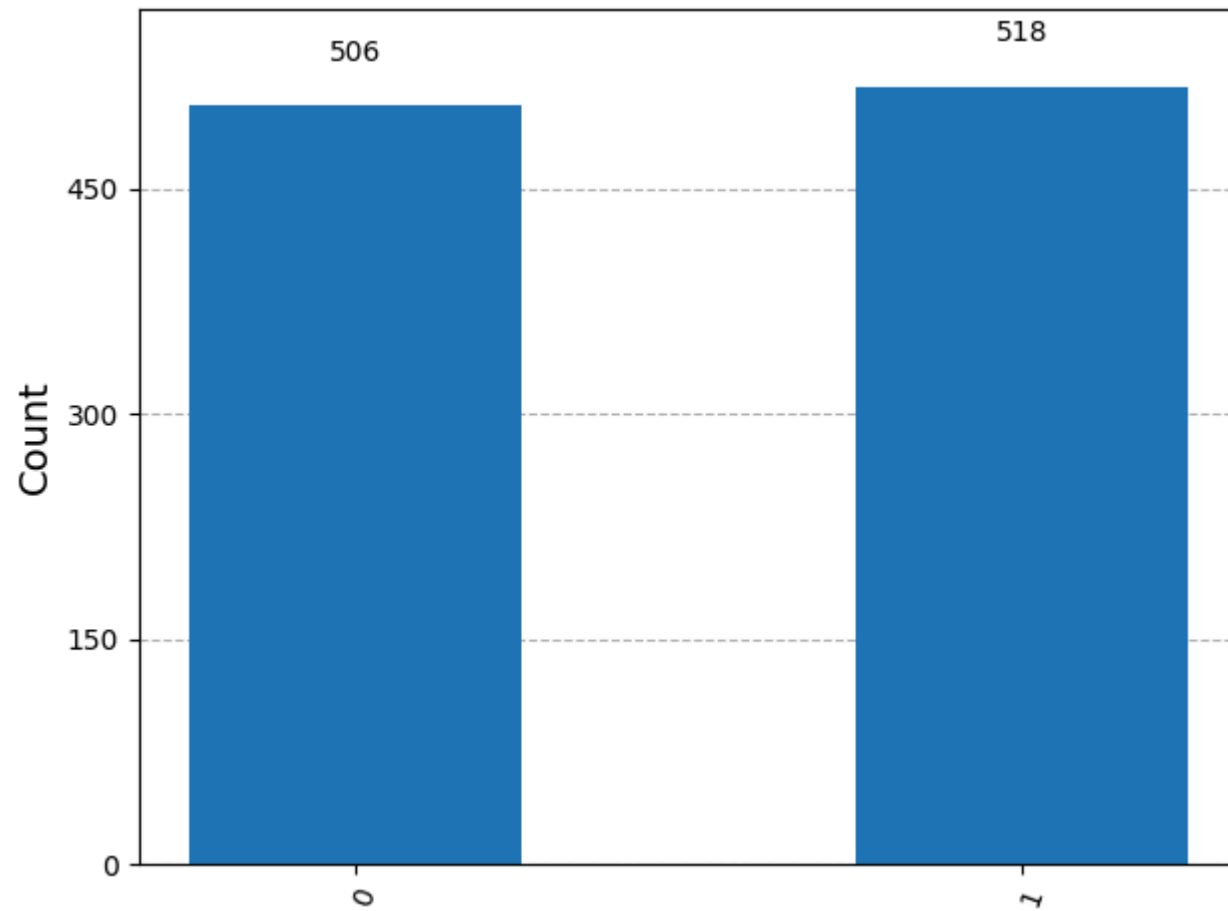
```
In [12]: from qiskit import generate_preset_pass_manager
from qiskit_aer import AerSimulator
from qiskit_ibm_runtime import Session, SamplerV2 as Sampler, QiskitRuntimeService

aer_sim = AerSimulator()
# Run the sampler job locally using AerSimulator.
pm = generate_preset_pass_manager(backend=aer_sim, optimization_level=1)
isa_qc = pm.run(qc)

# execute the quantum circuit
sampler = Sampler(aer_sim)
job = sampler.run([isa_qc])
result = job.result()

In [13]: plot_histogram(result[0].data.meas.get_counts())
```

Out[13]:



ANSWER: d

The reset operation returns a qubit to state $|0\rangle$

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/guides/visualize-results>

Section 2: Visualize quantum circuits, measurements, and states

4. Which one of the following images is the output from the code below:

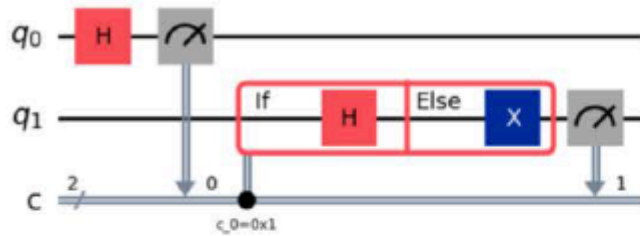
```
from qiskit import *

qubits = QuantumRegister(2)
clbits = ClassicalRegister(2)
circuit = QuantumCircuit(qubits, clbits)
(q0, q1) = qubits
(c0, c1) = clbits

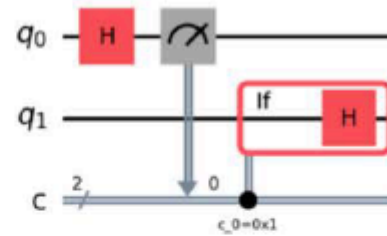
circuit.h(q0)
circuit.measure(q0, c0)
with circuit.if_test((c0, 1)) as else_:
    circuit.h(q1)
with else_:
    circuit.x(q1)
circuit.measure(q1, c1)

circuit.draw(output="mpl")
```

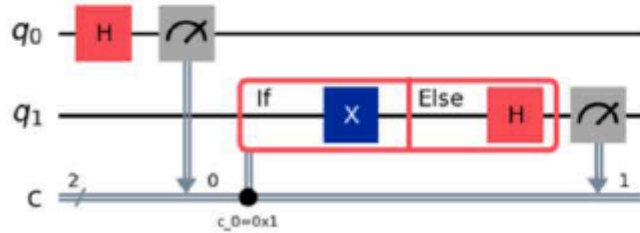
a.



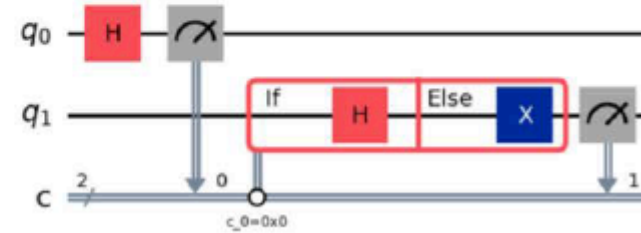
b.



c.

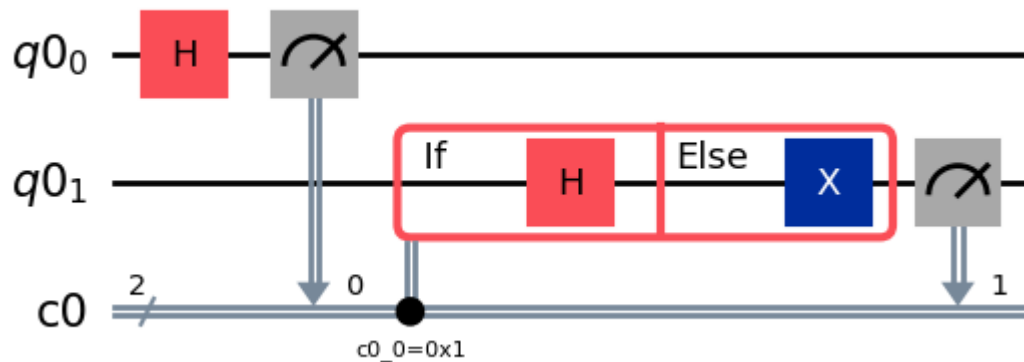


d.



```
In [14]: qubits = QuantumRegister(2)
clbits = ClassicalRegister(2)
circuit = QuantumCircuit(qubits, clbits)
(q0, q1) = qubits
(c0, c1) = clbits
circuit.h(q0)
circuit.measure(q0, c0)
with circuit.if_test((c0, 1)) as else_:
    circuit.h(q1)
with else_:
    circuit.x(q1)
circuit.measure(q1, c1)
circuit.draw(output="mpl")
```

Out[14]:



Dynamic circuits

- We can measure qubits in the middle of a quantum circuit execution
- Perform classical logic operations within the circuit, based on the outcome of those mid-circuit measurements.
- This process is also known as **classical feedforward**

Refer to the "**if statement**" in the reference link

- initialize registers with two qubits and two classical bits
- Apply a Hadamard gate to the Q0 and measure it
- If the result is 1, then we apply a Hadamard gate on the Q1
- otherwise, we apply an X gate on the second qubit.

Finally, we measure the second qubit as well.

ANSWER: a

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/guides/classical-feedforward-and-control-flow>

5. Given the code fragment below, which image is the expected output?

```
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram

state = Statevector([0.+0.j, 0.+0.j, 0.70710678+0.j, 0.+0.j,
                    0.+0.j, -0.70710678+0.j, 0.+0.j, 0.+0.j])
counts = state.sample_counts(shots=1024)
plot_histogram(counts)
```

REFERENCES:

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/visualization>

Bit order

<https://quantum.cloud.ibm.com/docs/en/guides/bit-ordering>

Plot quantum states

<https://quantum.cloud.ibm.com/docs/en/guides/plot-quantum-states>

NOTE: In Qiskit, bit $n-1$ is interpreted as the most significant bit.

Example:

$N=3$ (N = number of qubits)

No of states = $2^3 = 8$ possible basis states

Represents as binary strings from 000 to 111

q2--q1--q0

0---0---0

0---0---1

0---1---0

0---1---1

1---0---0

1---0---1

1---1---0

1---1---1

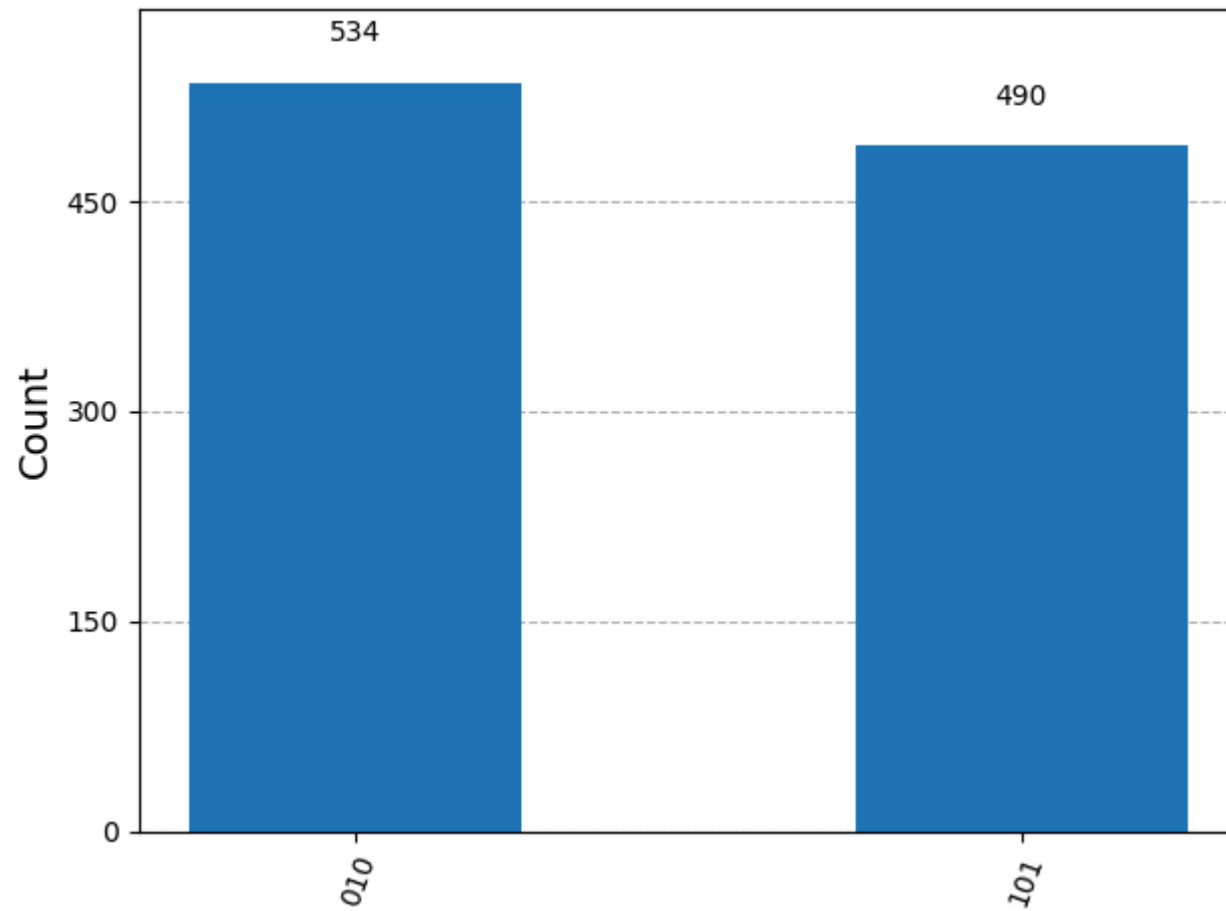
```
In [15]: from qiskit.quantum_info import Statevector
state = Statevector([0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125])
#would represent the state |000>
state.draw(output='latex')
```

Out[15]:

$$\frac{1}{8}|000\rangle + \frac{1}{8}|001\rangle + \frac{1}{8}|010\rangle + \frac{1}{8}|011\rangle + \frac{1}{8}|100\rangle + \frac{1}{8}|101\rangle + \frac{1}{8}|110\rangle + \frac{1}{8}|111\rangle$$

```
In [16]: state = Statevector([0.+0.j, 0.+0.j, 0.70710678+0.j, 0.+0.j,
0.+0.j, -0.70710678+0.j, 0.+0.j, 0.+0.j])
counts = state.sample_counts(shots=1024)
plot_histogram(counts)
```

Out[16]:



In [17]: `state.draw(output='latex')`

Out[17]:

$$\frac{\sqrt{2}}{2}|010\rangle - \frac{\sqrt{2}}{2}|101\rangle$$

ANSWER: c



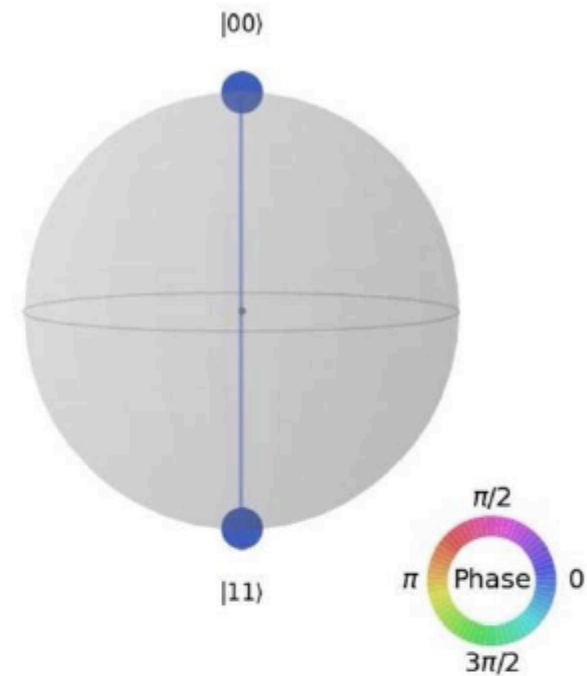
6. Which one of the following code fragments will generate the given qsphere representation visualization?

Note: the circles on the qsphere are the same color.

a. `qc = QuantumCircuit(2)`
`qc.h(0)`
`qc.z(0)`
`qc.cx(0, 1)`
`state = Statevector(qc)`
`plot_state_qsphere(state)`

b. `qc = QuantumCircuit(2)`
`qc.h(0)`
`qc.z(0)`
`qc.x(1)`
`qc.cx(0, 1)`
`state = Statevector(qc)`
`plot_state_qsphere(state)`

c. `qc = QuantumCircuit(2)`
`qc.x(1)`
`qc.h(0)`
`qc.cx(0, 1)`
`state = Statevector(qc)`
`plot_state_qsphere(state)`

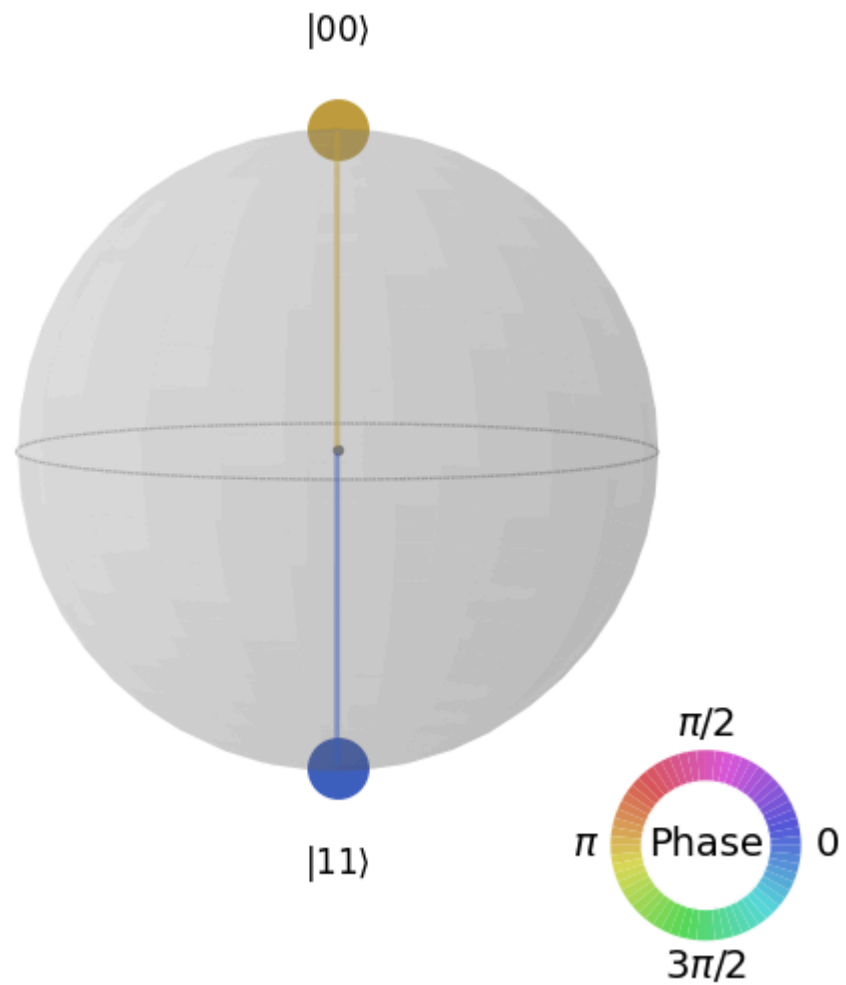


```
d. qc = QuantumCircuit(2)
   qc.h(0)
   qc.cx(0, 1)
   state = Statevector(qc)
   plot_state_qsphere(state)
```

Option a

```
In [18]: qc = QuantumCircuit(2)
         qc.h(0)
         qc.z(0)
         qc.cx(0, 1)
         state = Statevector(qc)
         plot_state_qsphere(state)
```

Out[18]:

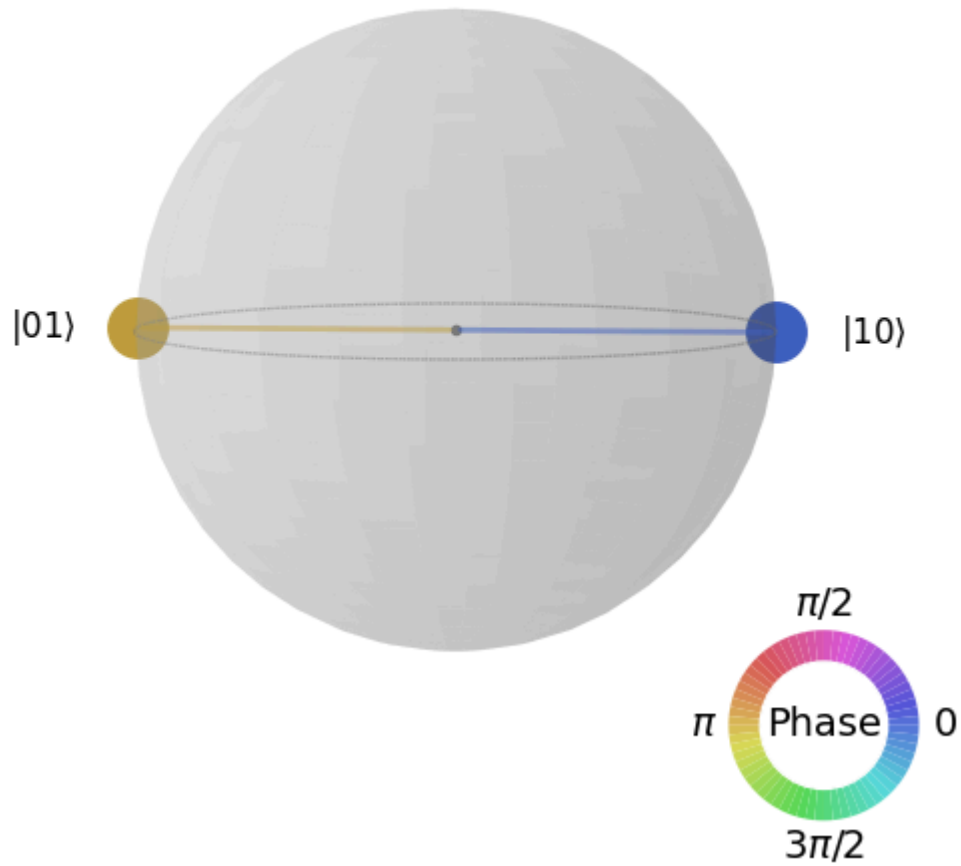


Option b

```
In [19]: qc = QuantumCircuit(2)
          qc.h(0)
          qc.z(0)
```

```
qc.x(1)
qc.cx(0, 1)
state = Statevector(qc)
plot_state_qsphere(state)
```

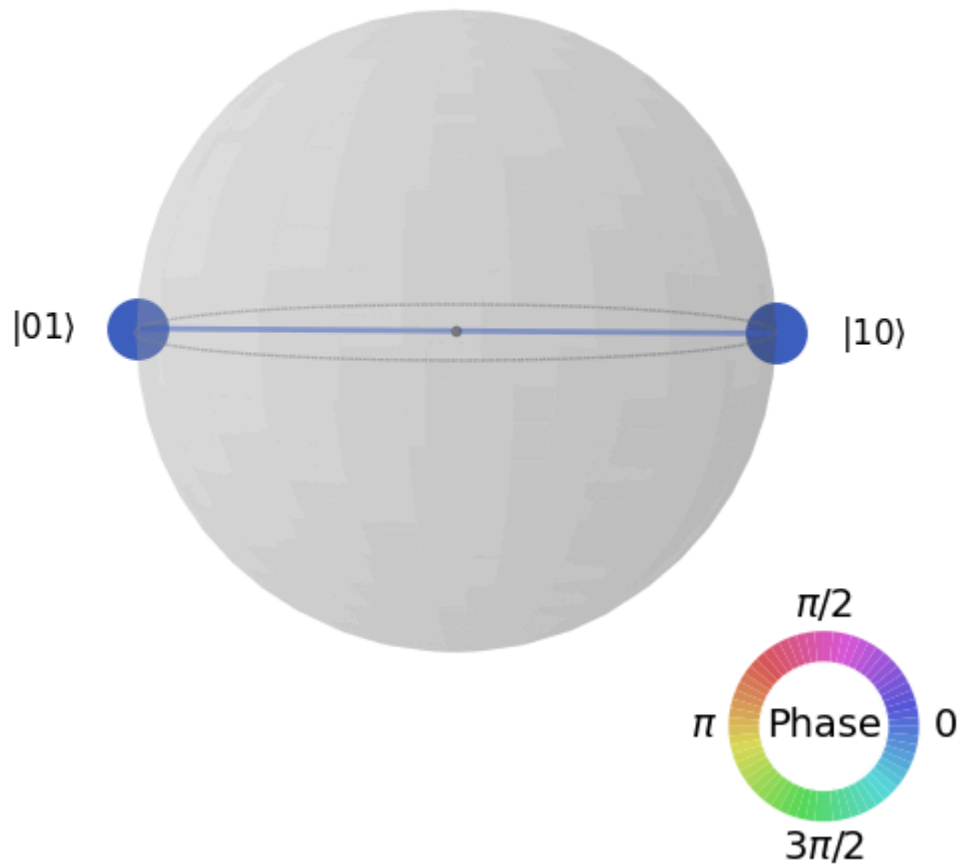
Out[19]:



Option c

```
In [20]: qc = QuantumCircuit(2)
         qc.x(1)
         qc.h(0)
         qc.cx(0, 1)
         state = Statevector(qc)
         plot_state_qsphere(state)
```

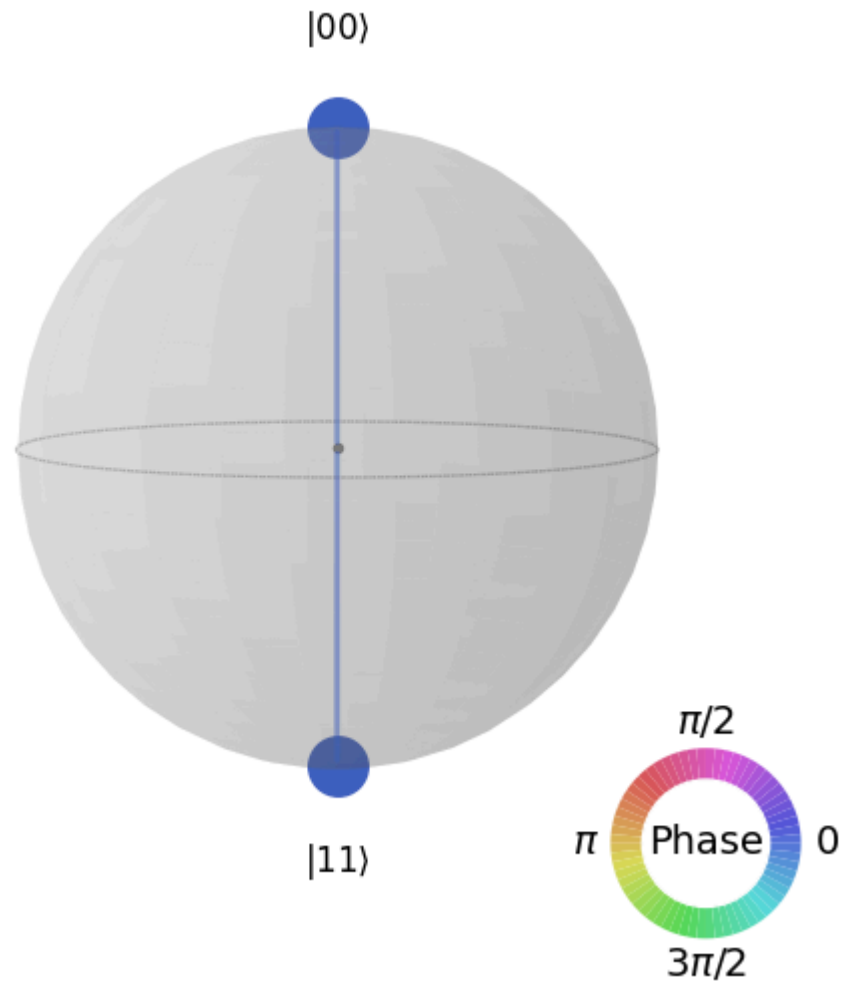
Out[20]:



ANSWER: d

```
In [21]: qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
state = Statevector(qc)
plot_state_qsphere(state)
```

Out[21]:



Section 3: Create quantum circuits

7. Given the code fragment below, which of the following code fragments creates a rotation gate with an angle with an initially undefined value?

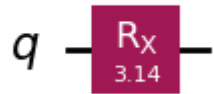
```
from qiskit.circuit import QuantumCircuit, Parameter,  
    ParameterExpression  
qc = QuantumCircuit(1)
```

- a. `theta = 3.14`
`qc.rx(3.14, 0)`
- b. `theta = Parameter('theta')`
`qc.rx(theta, 0)`
- c. `qc.rx('theta', 0)`
- d. `qc.rx(ParameterExpression('theta'), 0)`

Option a : Theta value is predefined

```
In [22]: qc = QuantumCircuit(1)  
theta = 3.14  
qc.rx(3.14, 0)  
qc.draw(output="mpl")
```

Out [22]:



Option b : Theta value is undefined

```
In [23]: qc = QuantumCircuit(1)
theta = Parameter('theta')
qc.rx(theta, 0)
qc.draw(output="mpl")
```

Out [23]:



Option c

```
qc.rx('theta', 0)
```

CircuitError: "Invalid param type <class 'str'> for gate rx."

Option d

```
qc.rx(ParameterExpression('theta'), 0)
```

TypeError: ParameterExpression.**init**() missing 1 required positional argument: 'expr'

ANSWER: b

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.ParameterExpression>

Additional Sample code to understand

```
In [24]: theta = Parameter('θ')
phi = Parameter('φ')

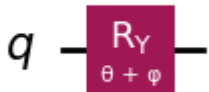
# Create a ParameterExpression
# This expression defines how the two parameters are combined
expr = theta + phi

# Create a quantum circuit
qc = QuantumCircuit(1)

qc.ry(expr, 0)

# Draw the circuit to see the symbolic expression
qc.draw(output="mpl")
```

Out [24]:



```
In [25]: # Bind concrete values to the parameters to get an executable circuit
bound_circuit = qc.assign_parameters({theta: 0.5, phi: 1.0})

# Draw the circuit to see the symbolic expression
bound_circuit.draw(output="mpl")
```

Out[25]:



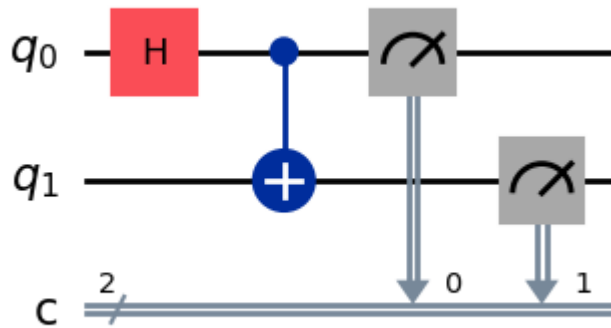
8. Which one of the following types of register stores the result of a measured circuit?

- a. Ancillary register
- b. Quantum register
- c. Classical register
- d. Circuit register

```
In [26]: from qiskit import QuantumCircuit

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])
qc.draw(output="mpl")
```

Out [26]:

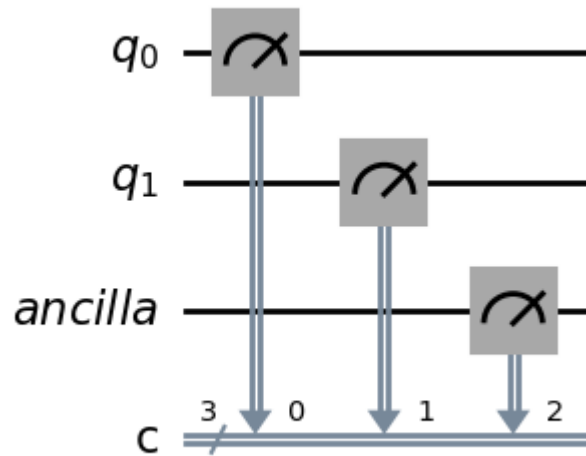


```
In [27]: print("Quantum Register info : ",qc.qregs)
print("Number of qubits: ",qc.num_qubits )
print("Classical Register info :", qc.cregs)
print("Number of classical bits :", qc.num_clbits)
```

```
Quantum Register info : [QuantumRegister(2, 'q')]
Number of qubits: 2
Classical Register info : [ClassicalRegister(2, 'c')]
Number of classical bits : 2
```

```
In [28]: qr = QuantumRegister(2, 'q')
anc = AncillaRegister(1, 'ancilla')
cr = ClassicalRegister(3, 'c')
qc = QuantumCircuit(qr, anc, cr)
qc.measure([0,1,2],[0,1,2])
qc.draw(output="mpl")
```

Out [28]:



```
In [29]: print("Quantum Register info : ",qc.qregs)
print("Number of qubits: ",qc.num_qubits )
print("Classical Register info :", qc.cregs)
print("Number of classical bits :", qc.num_clbits)
print("Ancilla Register info: ",qc.ancillas )
print("Number of Ancilla qubits in the circuit:", qc.num_ancillas)
```

```
Quantum Register info : [QuantumRegister(2, 'q'), QuantumRegister(1, 'ancilla')]
Number of qubits: 3
Classical Register info : [ClassicalRegister(3, 'c')]
Number of classical bits : 3
Ancilla Register info: [<AncillaQubit register=(1, "ancilla"), index=0>]
Number of Ancilla qubits in the circuit: 1
```

```
In [30]: print(qc.data)
```

```
[CircuitInstruction(operation=Instruction(name='measure', num_qubits=1, num_clbits=1, params=[]), qubits=(<Qubit reg
ister=(2, "q"), index=0>,), clbits=(<Clbit register=(3, "c"), index=0>,), CircuitInstruction(operation=Instruction
(name='measure', num_qubits=1, num_clbits=1, params=[]), qubits=(<Qubit register=(2, "q"), index=1>,), clbits=(<Clbi
t register=(3, "c"), index=1>,), CircuitInstruction(operation=Instruction(name='measure', num_qubits=1, num_clbits=
1, params=[]), qubits=(<AncillaQubit register=(1, "ancilla"), index=0>,), clbits=(<Clbit register=(3, "c"), index=2
>,,))]
```

ANSWER: b Classical Register

--- stores the result of our measured results
--- represents classical data in Qiskit

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.QuantumCircuit>

<https://quantum.cloud.ibm.com/docs/en/guides/measure-qubits>

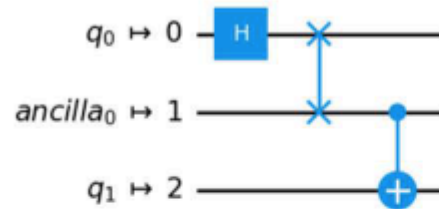
9. Given the code fragment below, which one of the following images could be produced?

```
from qiskit import QuantumCircuit
from qiskit import generate_preset_pass_manager

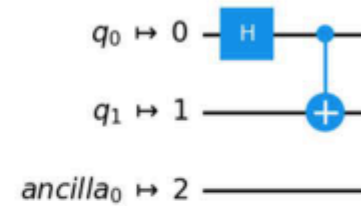
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)

pass_manager = generate_preset_pass_manager(
    optimization_level=3,
    coupling_map=[[0, 1], [1, 2]] ,
    basis_gates=['h', 'swap', 'cx'],
    initial_layout=[0, 2]
)
tqc = pass_manager.run(qc)
tqc.draw(output="mpl")
```

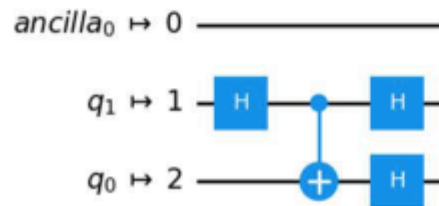
a.



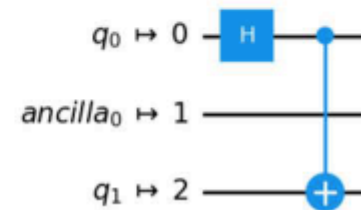
b.



c.



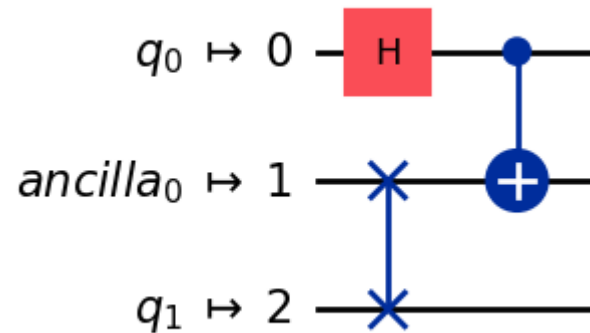
d.



```
In [31]: from qiskit import QuantumCircuit
from qiskit import generate_preset_pass_manager
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)

pass_manager = generate_preset_pass_manager(
    optimization_level=3,
    coupling_map=[[0, 1], [1, 2]] ,
    basis_gates=['h', 'swap', 'cx'],
    initial_layout=[0, 2]
)
tqc = pass_manager.run(qc)
tqc.draw(output="mpl")
```

Out[31]:



ANSWER: a

REFERENCE :

https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.transpiler.generate_preset_pass_manager#

Section 4: Run quantum circuits

10. Which three of the following are job execution modes in Qiskit Runtime?

- a. classical
- b. session
- c. parallel
- d. quantum
- e. batch
- f. single job

ANSWER: b,e,f

- job : A single primitive request of the estimator or the sampler made without a context manager
- session : A dedicated window for running a multi-job workload
- batch : A multi-job manager for efficiently running experiments comprising multi-job workloads

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/guides/execution-modes>

11. Which code fragment is the correct way to open a session?

- a.

```
from qiskit_ibm_runtime import Session
session = Session(system='ibm_foo')
```
- b.

```
from qiskit_ibm_runtime import execute, QiskitRuntimeService
service = QiskitRuntimeService()
session = execute(service=service)
```
- c.

```
from qiskit_ibm_runtime import Session, QiskitRuntimeService
service = QiskitRuntimeService()
session = Session(service.least_busy())
```
- d.

```
from qiskit import QuantumCircuit
session = QuantumCircuit(2).open_session()
```

Use sessions when you need dedicated and exclusive access to the QPU.

Before starting a session, you must set up Qiskit Runtime and initialize it as a service:

```
from qiskit_ibm_runtime import ( QiskitRuntimeService, Session )
```

```
service = QiskitRuntimeService()
```

Initialize the Session class.

To start a session, you must specify a QPU by passing a backend object

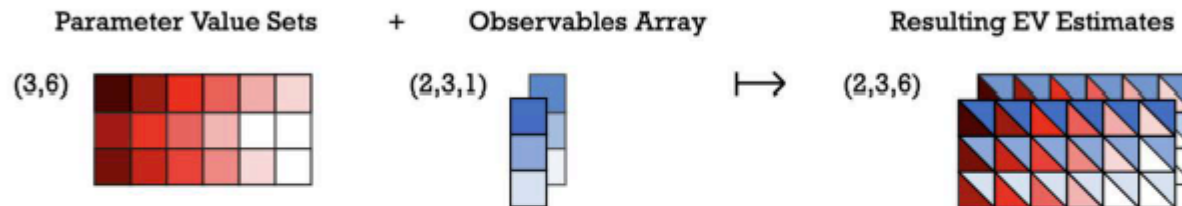
```
backend = service.least_busy(operational=True, simulator=False)
```

```
session = Session(backend=backend)
```

ANSWER: c

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/guides/run-jobs-session>

12. Which one of the following patterns, expressed in terms of array broadcasting primitives, is represented by the given image?

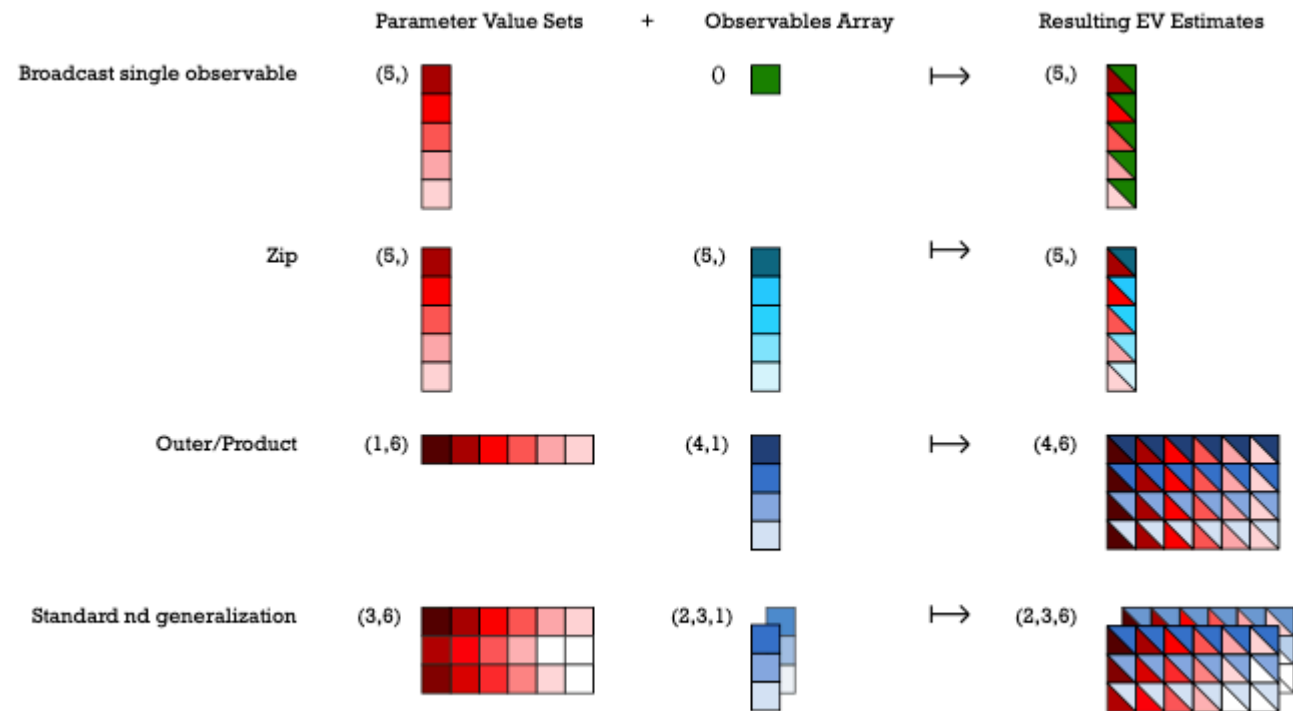


- a. Standard multidimensional array generalization
 - b. All-to-all
 - c. Extended dimensional variation
 - d. Best effort broadcasting
-

common patterns expressed in terms of array broadcasting

Parameter value sets are represented by $n \times m$ arrays, and observable arrays are represented by one or more single-column arrays

the parameter value sets are combined with their observable array to create the resulting expectation value estimates.



Visual representation of broadcasting

ANSWER: a

- **Standard nd generalization**
- 3x6 parameter value set array
- two 3x1 observables array
- These combine to create two 3x6 output arrays

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/guides/primitive-input-output>

Section 5: Use the sampler primitive

13. Given the following code fragment, which one of the following describes the `SamplerOptions` `parameter` `options.default_shots`?

```
...  
from qiskit_ibm_runtime import Sampler  
  
sampler = Sampler(mode=backend)  
sampler.options.default_shots = ...
```

- a. The sum of the number of measurements in each qubit
 - b. The number of randomizations we apply to the circuit
 - c. The number of times that we run the circuit
 - d. The number of sequences in dynamical decoupling
-

ANSWER: c

REFERENCE :

<https://quantum.cloud.ibm.com/docs/en/api/qiskit-ibm-runtime/options-sampler-options>

<https://quantum.cloud.ibm.com/docs/en/guides/runtime-options-overview>

The total number of measurements across all qubits is then simply the number of shots multiplied by the number of qubits.

Parameters

1. max_execution_time
2. environment
3. simulator
4. default_shots
5. dynamical_decoupling
6. execution
7. twirling
8. experimental

14. Given the code snippet, which one of the following is a valid way to invoke the run method on an instance of SamplerV2?

```
from qiskit_ibm_runtime import SamplerV2
...
```

```
sampler = SamplerV2(...)
```

- a. `sampler.run([isa_circuit])`
- b. `sampler.run(distribution, isa_circuit)`
- c. `sampler.run(isa_circuit, distribution='gauss')`
- d. `sampler.run([isa_circuit1, isa_circuit2], runs=1024)`

- Note: Any quantum circuit passed to a Sampler must include measurements

Example 1: execute 1 circuit with Sampler

```
job = sampler.run([circuit])
```

Example 2: Sample two circuits at 128 shots each.

```
sampler.run([isa_circuit1, isa_circuit2], shots=128)
```

Example 3: Sample two circuits at different amounts of shots.

The "None"s are necessary as placeholders for the lack of parameter values in this example.

```
sampler.run([(isa_circuit1, None, 123), (isa_circuit2, None, 456)])
```

ANSWER: a

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/guides/simulate-with-qiskit-sdk-primitives>

Section 6: Use the estimator primitive

15. Which one of the following describes the expected behavior of the number of shots if the value for the parameter `precision` were changed from `0.015625` to `0.03125`?

- a. It increases the number of shots quadratically
- b. It increases the number of shots exponentially
- c. It has no effect on the number of shots
- d. It decreases the number of shots

- Precision is used with reference to "Estimator only"
- Precision scales inversely with usage.
- That is, the lower the precision, the more QPU time it takes to run.

In [32]: `0.015625 < 0.03125`

Out[32]: True

- Value for the parameter precision were changed from 0.015625 to 0.03125
- Indicates a decrease in the desired precision

0.03125 is a larger value--> error is larger.

1. Decreasing precision-->larger value: leads to fewer shots
2. Increasing precision -->smaller value: leads to more shots

To achieve the increased accuracy, the primitive must run more shots or execute the circuit more times to get statistically significant results.

Summary

- the value 0.015625 is small --> Higher precision requires more shots
- the value 0.03125 is large --> lower precision requires few shots

ANSWER: d

increasing the precision value , requires less number of shots

REFERENCE: <https://quantum.cloud.ibm.com/docs/en/guides/specify-runtime-options>

16. Which error mitigation technique can be applied using resilience options?

- a. Pauli twirling
- b. Dynamical decoupling
- c. Zero Noise Extrapolation
- d. Full quantum error correction

Resilience levels

- 1. level 0 turns off all error mitigation
- 2. level 1 turns on options for measurement error mitigation, and
- 3. level 2 turns on options for gate and measurement error mitigation.

Resilience options for V2 Estimator.

- 1. measure_mitigation
- 2. measure_noise_learning
- 3. zne_mitigation
- 4. zne
- 5. pec_mitigation
- 6. pec
- 7. layer_noise_learning
- 8. layer_noise_model

ANSWER: c , ZNE

ZNE can be enabled by setting `zne_mitigation` to `True` in the Qiskit Runtime resilience options for Estimator

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/guides/specify-runtime-options>

<https://quantum.cloud.ibm.com/docs/en/api/qiskit-ibm-runtime/options-resilience-options-v2>

17. Which format should a primitive unified bloc (PUB) tuple follow for the Estimator primitive?

- a. `pub = (circuit, observable, parameter_values, backend)`
- b. `pub = (circuit, observable, parameter_values, precision)`
- c. `pub = (circuit, observable, shots, optimization_level)`
- d. `pub = (circuit, observable, resilience_level, noise_model)`

ANSWER: b

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/guides/primitive-input-output>

NOTE:

Estimator PUB

estimator_pub = (circuit, observables, parameter_values)

(Optionally) a target precision for expectation values to estimate

Sampler PUB

sampler_pub = (circuit, parameter_values)

(Optionally) a number of shots to measure the circuit with

Section 7: Retrieve and analyze the results of quantum circuits

18. Which statement describes the purpose of a Qiskit Runtime session?

- a. Automatically generate quantum algorithms based on user input
- b. Visualise the results of quantum experiments in real time
- c. Group a collection of calls to the quantum computer
- d. Compile and optimise quantum circuits for different backends

- Helps to run a multi-job workload
- run multiple experiments simultaneously
- allows users to experiment with variational algorithms
- Useful for iterative tasks , which requires frequent communication between classical and quantum resources

ANSWER: c

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/guides/execution-modes>

19. Which two of the following pieces of information are part of the dictionary returned by `session.details()`, assuming that `session` is an instance of `qiskit_ibm_runtime.Session`?

- a. Quantum circuit depth
- b. Timestamp of the last job in the session that completed
- c. Session state
- d. Primitive options
- e. Primitive unified blocs (PUBs) in each job

```
In [33]: '''
from qiskit_ibm_runtime import (
    QiskitRuntimeService,
    Session,
    EstimatorV2 as Estimator,
)

service = QiskitRuntimeService()
backend = service.least_busy(operational=False, simulator=True)

with Session(backend=backend) as session:
    print(session.details())
'''

# Note: not available to open plan users
```



```
Out[33]: '\nfrom qiskit_ibm_runtime import (\n    QiskitRuntimeService,\n    Session,\n    EstimatorV2 as Estimator,\n)\n\nservice = QiskitRuntimeService()\nbackend = service.least_busy(operational=False, simulator=True)\n\nwith Session(backend=backend) as session:\n    print(session.details())\n'
```

ANSWER: b,c

REFERENCE:

<https://quantum.cloud.ibm.com/docs/en/guides/run-jobs-session>

Section 8: Operate with OpenQASM

20. Which one of the following is a classical data type supported by OpenQASM 3?

- a. complex
- b. class
- c. char
- d. enum

- Latest version: OpenQASM 3
 - The OpenQASM 3.0 specification only directly permits complex numbers with floating-point component types
-

ANSWER: a

Reference:

<https://quantum.cloud.ibm.com/docs/en/guides/introduction-to-qasm>

<https://openqasm.com/versions/3.0/language/types.html>

21. Which method should be used to export a Qiskit circuit named `qc` to OpenQASM 3 and store it into a file stream named `qasmprogram`?

- a. `qc.to_openqasm3(qasmprogram)`
- b. `qiskit.qasm3.dump(qc, qasmprogram)`
- c. `qasmprogram.export_to_qasm3(qc)`
- d. `qiskit.qasm3.export(qc, qasmprogram)`

Export to OpenQASM 3

Export Qiskit code to OpenQASM 3 with `dumps()`, which exports to a string, or `dump()`, which exports to a file.

ANSWER: b

REFERENCE:

Converts Quantum circuit into qasm string

```
In [34]: from qiskit import QuantumCircuit
         from qiskit.qasm3 import dumps

         qc = QuantumCircuit(2)
         qc.h(0)
         qc.cx(0, 1)
         qc.measure_all()

         qasmprogram=dumps(qc)
         print(qasmprogram)
```

```
OPENQASM 3.0;
include "stdgates.inc";
bit[2] meas;
qubit[2] q;
h q[0];
cx q[0], q[1];
barrier q[0], q[1];
meas[0] = measure q[0];
meas[1] = measure q[1];
```

Converts Quantum circuit into qasm string and stores into a file

```
In [35]: from qiskit import QuantumCircuit
         from qiskit.qasm3 import dump

         qc = QuantumCircuit(2)
         qc.h(0)
         qc.cx(0, 1)
         qc.measure_all()
```

```
qasmprogram = open("my_file.txt", "w")
dump(qc, qasmprogram)
qasmprogram.close()
```

Load the QASM into a circuit

```
In [38]: from qiskit.qasm3 import loads
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

qasmprogram=dumps(qc)
print(qasmprogram)
circuit = loads(qasmprogram)
circuit.draw(output="mpl")
```

```
OPENQASM 3.0;
include "stdgates.inc";
bit[2] meas;
qubit[2] q;
h q[0];
cx q[0], q[1];
barrier q[0], q[1];
meas[0] = measure q[0];
meas[1] = measure q[1];
```

Out[38]:

